

TITLE:FRAUD DETECTION USING MACHINE LEARNING

SOURCE CODE:

```
Fraud-Detection-ML/
|
├── dataset/
│   └── creditcard.csv
|
├── model/
│   └── fraud_model.pkl
|
├── train_model.py
├── predict.py
├── app.py
├── templates/
│   └── index.html
├── static/
│   └── style.css
├── requirements.txt
└── README.md
```

1.train_model.py

```
import pandas as pd
import pickle

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("dataset/creditcard.csv")

# Features and Target
X = data.drop("Class", axis=1)
y = data["Class"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```

y,
test_size=0.2,
random_state=42,
stratify=y
)

# Train Model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

# Test Accuracy
prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

print("Model Accuracy :", accuracy)

# Save model
pickle.dump(model, open("model/fraud_model.pkl", "wb"))

print("Model Saved Successfully")

```

2. predict.py

```

import pickle
import numpy as np

model = pickle.load(open("model/fraud_model.pkl", "rb"))

def predict_transaction(features):

    features = np.array(features).reshape(1, -1)

    result = model.predict(features)

    if result[0] == 1:
        return "Fraud Transaction"

```

```
else:  
    return "Legitimate Transaction"
```

3. app.py

```
from flask import Flask, render_template, request  
import pickle  
import numpy as np  
  
app = Flask(__name__)  
  
model = pickle.load(open("model/fraud_model.pkl", "rb"))  
  
@app.route("/")  
def home():  
    return render_template("index.html")  
  
@app.route("/predict", methods=["POST"])  
  
def predict():  
  
    values = []  
  
    for i in range(30):  
        values.append(float(request.form[f"f{i}"]))  
  
    final = np.array(values).reshape(1, -1)  
  
    prediction = model.predict(final)  
  
    if prediction[0] == 1:  
        output = "Fraudulent Transaction"  
  
    else:  
        output = "Legitimate Transaction"  
  
    return render_template("index.html", prediction_text=output)  
  
if __name__ == "__main__":  
    app.run(debug=True)
```

4. templates/index.html

```
<!DOCTYPE html>

<html>

<head>

<title>Fraud Detection</title>

<link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

</head>

<body>

<div class="container">

<h1>Credit Card Fraud Detection</h1>

<form action="/predict" method="post">

{% for i in range(30) %}

<input type="text"
name="f{{i}}"
placeholder="Feature {{i}}"
required>

{% endfor %}

<br>

<button type="submit">Predict</button>

</form>

<h2>

{{ prediction_text }}
```

```
</h2>
```

```
</div>
```

```
</body>
```

```
</html>
```

5. static/style.css

```
body{
```

```
background:#f0f2f5;
```

```
font-family:Arial;
```

```
}
```

```
.container{
```

```
width:700px;
```

```
margin:auto;
```

```
background:white;
```

```
padding:30px;
```

```
margin-top:40px;
```

```
border-radius:10px;
```

```
box-shadow:0px 0px 10px gray;
```

```
}
```

```
h1{
```

```
text-align:center;
```

```
}
```

```
input{  
  
width:95%;  
  
padding:10px;  
  
margin:5px;  
  
}  
  
button{  
  
padding:12px;  
  
width:100%;  
  
background:#0b7dda;  
  
color:white;  
  
font-size:18px;  
  
border:none;  
  
cursor:pointer;  
  
}  
  
button:hover{  
  
background:#005fa3;  
  
}  
  
h2{  
  
text-align:center;  
  
color:green;  
  
}
```

6. requirements.txt

Flask
pandas
numpy
scikit-learn
pickle-mixin

Install using:

```
pip install -r requirements.txt
```

7. README.md

Credit Card Fraud Detection Using Machine Learning

Description

This project detects fraudulent credit card transactions using a Random Forest Classifier.

Dataset

Kaggle Credit Card Fraud Detection Dataset

creditcard.csv

Technologies

Python

Flask

Pandas

NumPy

Scikit-learn

HTML

CSS

Project Workflow

Load Dataset



Preprocess Data



Train Random Forest Model



Save Model



Load Model



Predict Transaction



Display Result

Run Project

Train Model

python train_model.py

Run Flask App

python app.py

Open Browser:

<http://127.0.0.1:5000>

How to Run

1. Download the **creditcard.csv** dataset from Kaggle and place it in the **dataset/** folder.
2. Install the required packages:

```
pip install -r requirements.txt
```

3. Train the model:

```
python train_model.py
```

This creates **model/fraud_model.pkl**.

4. Start the Flask application:

```
python app.py
```

5. Open your browser and go to:

```
http://127.0.0.1:5000
```